

Claude para alumnos de Negocios Digitales

Una guía intermedia para entender qué estás usando

Analítica de Datos · UdeSA · Otoño 2026 · Ignacio Aracena + Juan Costa

Por qué este material

Si estás en esta materia ya usaste un chatbot al menos una vez. Probablemente ChatGPT, capaz Gemini, capaz Claude. Lo usaste para resumir un texto, traducir algo, escribir un mail, debuggear código de la práctica.

El problema es que **usar la herramienta no es lo mismo que entenderla**. Y la diferencia importa: el alumno que entiende qué hace el modelo por dentro escribe mejores prompts, identifica cuándo el modelo está alucinando, sabe cuándo le conviene un agente y cuándo un chatbot, y termina diferenciándose en el mercado de trabajo.

Este PDF asume que ya tuviste contacto con un chatbot. Te lleva un par de niveles más adentro: qué es exactamente Claude, cómo funciona, en qué se diferencia de ChatGPT y Gemini, qué significa que sea un "agente", y cómo aprovecharlo bien para esta materia y para tu carrera.

Hay dos roadmaps al final: uno corto para Analítica de Datos (4 semanas) y otro extendido para quien quiera ir más profundo (3-6 meses).

1. Qué es Claude

Anthropic en una página

Claude es el modelo de IA desarrollado por **Anthropic**, una compañía fundada en 2021 por ex-investigadores de OpenAI (Dario y Daniela Amodei, entre otros). Anthropic se posiciona distinto a la competencia:

- **OpenAI** (ChatGPT) — el más popular. Foco en producto masivo y empresas grandes.
- **Google DeepMind** (Gemini) — integrado al ecosistema Google (Search, Workspace, Android).
- **Anthropic** (Claude) — foco declarado en **AI safety** y razonamiento. Pelea por el segmento de developers, empresas que necesitan modelos confiables, y casos de uso agentic.

Anthropic recibe inversión de Amazon (Anthropic vive principalmente sobre AWS) y Google. Es la segunda compañía de IA más grande del mundo después de OpenAI por valuación, en una carrera donde el tablero cambia cada trimestre.

La familia de modelos: Opus, Sonnet, Haiku

Cuando se habla de "Claude" no se está hablando de un modelo único. Es una familia. Cada generación tiene tres tamaños con el mismo nombre y diferente capacidad:

Modelo	Para qué sirve	Costo relativo
Opus	El más capaz. Razonamiento complejo, código difícil, análisis largos. Para problemas serios.	Caro
Sonnet	El equilibrado. La mayoría de las tareas profesionales caen acá: redacción, análisis de datos, código de complejidad media.	Medio
Haiku	El rápido y barato. Para volumen alto: clasificación, resumen, atención al cliente.	Bajo

A noviembre de 2025 la generación más nueva es **Claude 4**, con versiones como Opus 4.7 y Sonnet 4.6. Cuando entrás a claude.ai por default usás el modelo Sonnet — si querés cambiar, lo elegís desde el selector arriba a la izquierda.

Heurística práctica: para resolver una práctica de Analítica de Datos, Sonnet alcanza. Para un problema realmente difícil donde Sonnet se traba o alucina, escalá a Opus. Para tareas mecánicas repetitivas, bajá a Haiku.

El posicionamiento: safety, reasoning, agentic

Las tres palabras que más se repiten en Anthropic:

- **Safety** — comportamiento honesto, rechazo elegante de pedidos dañinos, baja tendencia a inventar respuestas. Es por lo que muchas empresas grandes en sectores regulados (finanzas, salud, legal) eligen Claude.

- **Reasoning** — capacidad de pensar paso a paso antes de responder. Claude tiene modos de "extended thinking" donde el modelo razona internamente antes de generar la respuesta visible.
- **Agentic** — diseñado para usarse como agente, no solo como chatbot. Esto se ve en cosas como Claude Code: el modelo decide qué herramientas usar, las usa, evalúa el resultado, y sigue trabajando.

2. Cómo funciona Claude (lo mínimo para entenderlo)

No hace falta saber matemática para usarlo bien, pero entender los fundamentos te ahorra muchos errores.

Tokens y context window

Claude no lee "palabras". Lee **tokens**. Un token es un fragmento de texto: a veces es una palabra entera, a veces parte de una. En español, una palabra ocupa en promedio 1.5 tokens. "Analítica" puede ser 3 tokens (`Anal` , `ít` , `ica`).

El **context window** es la cantidad máxima de tokens que el modelo puede procesar de una vez (tu prompt + lo que ya conversaron + la respuesta). Los modelos actuales de Claude tienen ventanas enormes — Opus 4.7 maneja **1 millón de tokens** (aprox. 700.000 palabras, o sea ~1.500 páginas de un libro).

Implicaciones prácticas:

- Podés pegarle un dataset entero, una clase magistral completa y un PDF de teoría, y Claude los puede mirar simultáneamente.
- A medida que la conversación se alarga, parte del contexto temprano puede empezar a perderse de la "atención" del modelo. Por eso conviene cortar conversaciones largas y abrir una nueva con contexto refrescado.

Predicción de siguiente token

Por dentro, Claude (como ChatGPT, Gemini, etc.) hace una sola cosa: **predecir el próximo token más probable** dado todo lo que ya vio. Lo hace una vez, agrega el token a lo que tiene, y vuelve a predecir el siguiente. Así genera respuestas palabra por palabra.

Esto suena simple pero tiene una consecuencia central:

El modelo no "sabe" cosas. Aprendió patrones de texto.

Cuando le preguntás "¿cuál fue el PBI de Argentina en 2023?", no consulta una base de datos. Genera la respuesta más probable según los patrones que aprendió durante el entrenamiento. Si los datos del entrenamiento son confiables y consistentes, la respuesta es buena. Si los patrones que aprendió son confusos o el dato es muy específico, **inventa** una respuesta plausible. Esto se llama **alucinación**.

Por qué alucina y cómo evitarlo

Claude alucina menos que otros modelos (parte del foco en safety), pero no es inmune. Patrones donde es más probable que invente:

- Estadísticas específicas con números (PBI, fechas, precios).
- Citas textuales atribuidas a personas.
- APIs, librerías o funciones que parecen existir pero no.
- Detalles muy específicos de un caso particular (la cláusula 7 del contrato laboral argentino, por ejemplo).

Cómo evitar caer en alucinaciones:

1. **Dale contexto explícito.** Si necesitás trabajar sobre un PDF, pegalo en el prompt. No le pidas que "recuerde" lo que dice un libro.
2. **Pedile fuentes.** Si pide un número, pedile que diga de dónde lo sacó. Si no puede, desconfiá.
3. **Activá búsqueda web** cuando sea posible. Claude.ai tiene un toggle de web search que reduce alucinaciones para datos actuales.
4. **Verificá lo crítico manualmente.** Para una práctica de la materia alcanza con cross-check. Para una decisión de negocio real, siempre.

Temperatura y determinismo

La **temperatura** es un parámetro técnico (de 0 a 1, aproximadamente) que controla cuán "creativo" es el modelo:

- Temperatura 0 — la respuesta más probable. Más determinista. Útil para tareas técnicas: código, traducciones, clasificación.
- Temperatura alta (0.7-1) — más variedad. Útil para tareas creativas: brainstorming, redacción.

En claude.ai la temperatura está fija (no la podés tocar). En la API y en Claude Code se puede ajustar. Por default, los productos finales usan una temperatura intermedia que funciona bien en la mayoría de casos.

3. Claude vs ChatGPT vs Gemini

Las tres familias de modelos resuelven en general las mismas tareas. Lo que las diferencia es **estilo, prioridades y ecosistema**.

Aspecto	Claude (Anthropic)	ChatGPT (OpenAI)	Gemini (Google)
Foco principal	Safety + reasoning + agentic	Producto masivo + ecosistema developer	Integración con Google Workspace + multimodal
Brillan en	Tareas de razonamiento, código, escritura técnica, análisis largo	UX masivo, voz, generación de imágenes, plugins	Búsqueda actualizada, integración Gmail/Drive/Docs
Costo	Medio-alto. Plan pago desde USD 20/mes	Medio. Plan Plus USD 20/mes	Bajo. Plan pago USD 20/mes, parte gratis vía Google AI Studio
Modos de uso	Chat (claude.ai) + API + Claude Code (agentic)	ChatGPT + API + Operator/Agents	Gemini App + API + Gemini en Workspace
Open source	No	No	Parcialmente (Gemma)

Cuándo conviene cada uno (opinión honesta):

- **Claude** — escritura larga, análisis de documentos extensos, código complejo, agentes autónomos (Claude Code), tareas donde la precisión importa más que la creatividad pura.
- **ChatGPT** — generación rápida, uso casual, voz, imágenes, plugins/agents.
- **Gemini** — cualquier cosa que requiera datos actualizados en tiempo real (Gemini consulta Google Search nativamente), trabajos sobre tu propio Gmail/Drive.

Para Analítica de Datos en esta materia, **Claude es el que mejor performa** en código de pandas/sklearn y en explicar resultados de modelos. Por eso es el que usamos. ChatGPT y Gemini también funcionan, pero los notebooks de la materia se diseñaron alrededor del estilo de respuesta de Claude.

4. Qué es un agente (y por qué importa)

Chatbot vs agente

Esta es probablemente la distinción más importante de todo el PDF.

Un **chatbot** es lineal: vos preguntás, el modelo responde, fin. Si la respuesta requiere "ir a buscar algo más", el chatbot no puede. Te dice "no tengo acceso a esa información" o inventa una respuesta plausible (alucina).

Un **agente** es un loop: el modelo recibe tu pedido, decide qué herramientas necesita usar (buscar en la web, leer un archivo, correr código, mandar un mail, llamar a una API), las usa, ve el resultado, decide qué hacer a continuación, y repite hasta cumplir el objetivo. No esperás cada paso — le decís "qué" querés que pase y el agente resuelve el "cómo".

Chatbot	Agente
1 turno: pregunta → respuesta	N turnos: piensa → actúa → observa → piensa de nuevo
Solo texto	Texto + acciones reales (archivos, comandos, APIs)
Pasivo: no decide nada por sí mismo	Activo: toma decisiones intermedias
Útil para preguntas puntuales	Útil para tareas multi-paso

El loop agentic

Un agente sigue un ciclo conocido como **ReAct** (Reason + Act):

1. **Piensa** qué necesita hacer.
2. **Elige una herramienta** disponible (leer archivo, correr código, buscar en web, etc.).
3. **Ejecuta** la herramienta y obtiene un resultado.
4. **Evalúa** si lo que obtuvo lo acerca al objetivo.
5. Si llegó, **responde**. Si no, **vuelve al paso 1**.

Este ciclo puede repetirse 5, 50 o 500 veces hasta que el agente termine. Por eso un agente puede armarte un análisis de datos completo (cargar el CSV, limpiarlo, entrenar 3 modelos, comparar resultados, generar gráficos, escribir un reporte) sin que vos intervengas en cada paso.

Ejemplos concretos para fijar la idea

Chatbot:

Vos: "¿Cuáles son los outliers en este dataset?"

Chatbot: "Sin ver el dataset no puedo decirte. Generalmente se detectan con IQR..."

Agente (Claude Code):

Vos: "Detectame outliers en `ventas.csv` y dame un boxplot."

Agente: lee el CSV → calcula IQR → identifica filas → genera el gráfico → te lo muestra → te explica el resultado.

Por qué Claude Code es agente y por qué eso cambia todo

Claude Code es la herramienta agentic oficial de Anthropic. No es solo un chatbot que sabe de código — es un agente completo que vive en tu terminal, lee tus archivos, corre comandos, edita código, abre branches de git, deploya, etcétera. Vos le decís "agregá tests al módulo de auth, arreglá los que fallen, y subí un PR", y se ocupa de los pasos sin que tengas que pedirle cada uno.

Para esta materia, Claude Code es lo más parecido a tener un asistente personal que sabe pandas, sklearn y matplotlib, y que ya leyó todos los notebooks del repo.

5. Claude Code

Qué es

Claude Code es una herramienta de Anthropic que pone un agente con Claude adentro, conectado a tu computadora y a tus archivos. Inicialmente fue pensado para desarrollo de software, pero hoy se usa también para análisis de datos, automatización personal y workflows de productividad.

Lo importante: **podés usar Claude Code sin saber programar**. Le hablás en español, él hace lo que sea necesario. Cuando algo del flujo requiere pensar (caso de negocio, decisión, criterio), te frena y te pregunta.

Surfaces (dónde podés correrlo)

Claude Code corre en varios lugares, todos sincronizados:

- **Terminal (CLI)** — el más potente. Para Mac, Linux y Windows.
- **Visual Studio Code** — extensión nativa con vista de diffs.
- **Desktop app** — app standalone para Mac y Windows. Tiene UI gráfica.
- **Web** — en claude.ai/code corre todo en la nube de Anthropic, no necesitás instalar nada.
- **JetBrains** (PyCharm, WebStorm, IntelliJ) — plugin oficial.
- **iOS app** — para tareas en el celular.

Para arrancar en esta materia, recomiendo **Desktop app o Web**. La CLI tiene más poder pero pide más manejo de terminal.

Cómo se instala

Web (sin instalar nada):

Andá a claude.ai/code, iniciá sesión con tu cuenta de Anthropic y listo.

Mac (Homebrew):

```
brew install --cask claude-code
```

Mac / Linux / WSL (script oficial):

```
curl -fsSL https://claude.ai/install.sh | bash
```

Windows (PowerShell):

```
irm https://claude.ai/install.ps1 | iex
```

Después, en cualquier carpeta:

```
cd mi-proyecto  
claude
```

La primera vez te pide loguearte con tu cuenta de Anthropic. Listo.

Cómo se usa

Una vez adentro:

```
> Analizá los datos del archivo ventas.csv y generame un reporte  
con los productos top y los meses pico.
```

Claude Code va a:

1. Leer el archivo.
2. Pensar qué columnas tiene.
3. Generar el código de análisis.
4. Correrlo.
5. Mostrarte los resultados con interpretación.

Si pide permiso para algo (borrar un archivo, correr un comando potencialmente destructivo), te pregunta primero. La filosofía es: **lee y explora con libertad, pero pide permiso para actuar**

sobre cosas importantes.

Tres conceptos que vale conocer

- **CLAUDE.md** — un archivo que ponés en la raíz de tu proyecto con instrucciones permanentes para Claude (tono, librerías favoritas, reglas del repo). Cada vez que abrís Claude Code en ese proyecto, lo lee. El repo de esta materia tiene su propio CLAUDE.md.
- **Skills** — workflows reutilizables que te armás vos. Por ejemplo, un skill `/limpiar-dataset` que siempre aplica los mismos pasos.
- **MCP** (Model Context Protocol) — la forma de conectar Claude a herramientas externas: Google Drive, Notion, bases de datos, etc. Si querés que Claude lea tus apuntes de Notion, hay un MCP que lo permite (ver sección 7).

6. Prompting y Context Engineering

Tipos de prompting (los nombres que se mencionan en cursos)

- **Zero-shot** — le pedís algo sin ejemplos. "Traducime este texto al inglés." Funciona para tareas que el modelo ya conoce bien.
- **Few-shot** — le das 2-3 ejemplos del formato que querés antes de pedirle lo nuevo. Funciona cuando el formato de salida importa.
- **Chain-of-Thought (CoT)** — le pedís explícitamente que "piense paso a paso" antes de dar la respuesta. Útil para problemas de razonamiento.
- **Role prompting** — le decís quién es ("sos un analista financiero senior..."). Funciona menos de lo que la gente cree, pero ayuda al tono.

Context Engineering (el reemplazo del prompt engineering)

La conversación sobre "prompt engineering" está evolucionando hacia algo más amplio: **context engineering**. La idea: lo que más mueve la aguja no es escribir frases mágicas en el prompt, sino **darle al modelo el contexto correcto**.

Buen context engineering:

1. **Le pasás los documentos relevantes** (PDFs, datasets, código) directamente en el prompt.
2. **Le decís quién sos y qué necesitás** (nivel técnico, audiencia final).
3. **Le aclarás restricciones** (longitud, formato, qué evitar).
4. **Iteración con feedback** (no esperás que acierte de primera; le corregís y refinás).

Plantilla mínima de prompt para esta materia:

[CONTEXTO]

Soy alumno de Analítica de Datos en UdeSA. Estamos viendo CART en clase 7. Tengo el dataset `netflix_suscriptores.csv` con 1500 filas.

[QUE NECESITO]

Quiero entender por qué mi árbol tiene profundidad 4 y no 6.

[FORMATO]

Explicame en 3-4 párrafos, sin código, con un ejemplo del impacto en mi caso.

Errores comunes que cometen los alumnos

1. **Prompts vagos** — "explicame esto" sin contexto. El modelo improvisa.
2. **Aceptar la primera respuesta** — el modelo te dio algo plausible, pero capaz no es lo mejor. Iterá: "dame 3 alternativas distintas a esto".
3. **No darle el archivo** — le pedís comentario sobre un código sin pegarle el código. El modelo inventa.
4. **Confiar en datos numéricos sin verificar** — siempre cross-check los números que te tira.
5. **Mezclar idiomas** — si querés respuesta en español, mantené el prompt en español. Si lo mezclás, la salida es inconsistente.

7. Tool Use y MCP

Function calling / Tool use

Los modelos modernos pueden **llamar a funciones que vos definís**. Le pasás una lista de herramientas disponibles (con su descripción y parámetros), y el modelo decide cuándo usar cada una.

Esto es lo que hace que un agente sea un agente: tiene herramientas que puede invocar. En Claude Code, las herramientas built-in incluyen:

- **Read** — lee un archivo.
- **Write** / **Edit** — crea o modifica un archivo.
- **Bash** — corre un comando de shell.
- **Grep** / **Glob** — busca archivos por patrón.
- **WebFetch** / **WebSearch** — busca en internet.

Cuando le pedís "limpiá los nulos del CSV", Claude Code combina estas herramientas: usa **Read** para ver el archivo, decide qué hacer, usa **Edit** para escribir el código de limpieza, usa **Bash** para correrlo, lee el resultado, te muestra el output.

MCP (Model Context Protocol)

MCP es un estándar abierto que Anthropic lanzó para que cualquier herramienta del mundo se pueda conectar a Claude (o a cualquier otro modelo que lo soporte). Es como un USB universal para conectar Claude a tus datos y aplicaciones.

Hay MCPs oficiales (de Anthropic) y comunitarios para:

- **Google Drive / Docs / Sheets / Calendar** — leer y editar.
- **Notion** — leer y editar páginas y bases de datos.
- **Slack** — buscar, leer, mandar mensajes.
- **GitHub** — leer repos, abrir PRs.
- **Jira / Asana / Linear** — gestionar tickets.
- **Bases de datos** (Postgres, MySQL, BigQuery) — consultar.

Para esta materia, los MCPs más útiles probablemente serían **Google Drive** (si guardás apuntes ahí) y **Notion** (si lo usás). La configuración es simple: una línea en un archivo de config de Claude Code.

Ejemplo realista

Un MCP de Google Drive te deja decir cosas como:

"Buscá en mi Drive los apuntes de clase 6 de Pablo, leélos, y armame un resumen para estudiar para el parcial."

Sin MCP, tendrías que copiar y pegar manualmente cada documento. Con MCP, Claude Code los busca y los procesa.

8. Cómo empezar HOY

Paso 1 — Cuenta y Claude.ai

1. Andá a <https://claude.ai> y creá una cuenta (sirve cuenta de Google).
2. Probá conversaciones simples para entender la interfaz.

3. Plan **gratuito** alcanza para empezar. Tiene límite de mensajes diarios pero es generoso.

Paso 2 — Claude Code

Para esta materia, recomiendo arrancar con la **versión web**: andá a claude.ai/code y empezá ahí. Sin instalación, sin terminal.

Cuando estés cómodo, pasate a la **Desktop app** o a la **CLI** para tener más control.

Plan gratuito vs pago

Plan	Costo	Para qué sirve
Free	\$0	Probar Claude.ai con límite diario de mensajes. Suficiente para curiosar.
Pro	USD 20/mes	Mensajes generosos, Sonnet por default, acceso a Claude Code. Lo razonable para un alumno que la quiera usar en serio.
Max	USD 100-200/mes	Volúmenes altos, prioridad, acceso a Opus por default. Para profesionales o quienes usan Claude Code intensivamente.
API	Pay-per-use	Para integrar Claude en tus propios programas. Pagás por token.

Para esta materia el **Pro** es lo recomendable si la usás varias veces por semana. Si solo curioséas, el Free alcanza.

Tips de seguridad

- **Nunca subas datos personales reales** a Claude.ai sin entender las políticas de privacidad. Si trabajás con un dataset sensible, anonimízalo antes.
- **No pegues passwords ni API keys** en el chat. Por más que sea Anthropic, el principio es no enviar lo que no querrías que se viera.
- **Activá 2FA** en tu cuenta de Anthropic.
- **Cuando uses Claude Code**, prestá atención cuando pide permisos antes de ejecutar acciones destructivas (borrar archivos, push a main, etc.). No autorices sin leer.

9. Roadmap A — Aplicado a Analítica de Datos (4 semanas)

Roadmap pensado para que en un mes seas independiente usando Claude para resolver esta materia.

Semana 1 — Cuenta y primer contacto

- Crear cuenta en Claude.ai.
- Conversaciones de prueba: pedile que explique un concepto del PDF de Pablo (KNN, CART, Regresión Logística).
- Pegale un notebook de la materia y pedile que te lo explique celda por celda.
- Pedile que te invente un quiz de 5 preguntas sobre el tema.

Semana 2 — Entendiendo y modificando código

- Pegale el código de una práctica y pedile que explique línea por línea las partes que no entendés.
- Modificar parámetros: "cambiá el `max_depth` de 4 a 6 y explicame qué efecto esperás".
- Detectar problemas: "este código me tira un warning, qué significa".

Semana 3 — Claude Code primera vez

- Bajar Claude Code Desktop o usar Web.
- Abrir el repo de la materia.
- Pedirle que ejecute el notebook de la clase actual.
- Pedirle que armé una versión modificada para un caso ficticio (otro dataset, otra empresa).

Semana 4 — Workflow propio

- Para una práctica futura, **planificar con Claude.ai** (qué pasos, qué métricas, qué decisión de negocio).
- **Ejecutar con Claude Code** (escribir código, correr, evaluar).
- **Pedile crítica honesta** ("¿qué problemas ves en este enfoque?").
- Armar tu propio CLAUDE.md de proyecto con tu estilo.

Al final del mes, tendrías que ser capaz de resolver una clase nueva de la materia en ~70% del tiempo que te llevaba antes, con mejores explicaciones de los resultados.

10. Roadmap B — Profundización 3-6 meses

Roadmap para alumnos que quieren ir más allá de "usar Claude" y entender bien la base. Esto te diferencia mucho en una entrevista laboral.

Mes 1 — Fundamentos de IA generativa

- **Anthropic Academy** (anthropic.skilljar.com) — empezá con los cursos:
- *Claude 101*
- *AI Fluency*
- *Claude Code in Action*
- Lee los [docs oficiales de Claude](#), especialmente la sección de prompting.
- Recurso recomendado: **Andrej Karpathy** en YouTube — "Intro to LLMs" y "Let's build the GPT tokenizer".

Mes 2 — Prompting y Context Engineering

- Curso de Anthropic Academy: *Prompt Engineering Interactive Tutorial*.
- Hacé tu propia colección de prompts útiles para tu carrera.
- Experimentá con few-shot y chain-of-thought en problemas reales.
- Lee el blog de Anthropic sobre **context engineering** (cambian la conversación de prompt engineering a context engineering).

Mes 3 — Tool Use, Sub-Agents y MCP

- Curso de Anthropic Academy: *Tool Use in Claude*.
- Configurá MCP en tu Claude Code: conectalo a tu Google Drive o Notion.
- Experimentá con [sub-agents](#) — un agente que lanza otros agentes en paralelo.
- Project: armate un workflow personal donde Claude te organice tu calendario, mails y tareas pendientes vía MCP.

Mes 4 — Evaluaciones y Production

- Aprendé a evaluar respuestas de un modelo: cuándo un modelo es "bueno" para tu caso.
- Lee sobre **prompt caching** y **batch processing** (cómo ahorrar plata en producción).
- Curso de Anthropic Academy: *Real World Prompting* y *Building with Extended Thinking*.

Mes 5-6 — Proyecto personal

- Elegí un problema real de tu carrera, vida o negocio (no académico).
- Diseñá una solución end-to-end con Claude (chat, código, agentes, MCP).
- Documentá el proceso: prompts que funcionaron, los que no, métricas.
- Compartilo (LinkedIn, blog, repo público). **Esto vale oro en un CV.**

11. Recursos curados

Documentación oficial

- <https://claude.ai> — la app de chat
- <https://claude.ai/code> — Claude Code en el navegador
- <https://docs.anthropic.com> — documentación del modelo y la API
- <https://code.claude.com/docs> — documentación específica de Claude Code

Cursos (todos gratis)

- [Anthropic Academy](#) — la fuente oficial. Cursos de Claude 101, AI Fluency, Claude Code in Action, Prompt Engineering, Tool Use, MCP, Building with Extended Thinking. Tienen certificación.
- [DeepLearning.AI — Building with Claude](#) — curso corto de Andrew Ng en colaboración con Anthropic.

Blogs y newsletters

- <https://www.anthropic.com/news> — anuncios oficiales y notas de investigación.
- <https://www.anthropic.com/research> — papers y reportes técnicos. Son legibles.

YouTube

- **Anthropic** (canal oficial) — videos cortos sobre features de Claude Code.
- **Andrej Karpathy** — el mejor explicador del mundo de cómo funcionan los LLMs por dentro.
- **AI Explained** — análisis serio de modelos y benchmarks.

Comunidades

- **Discord de Anthropic** (link en docs.anthropic.com) — comunidad oficial.
- **r/ClaudeAI** en Reddit.
- **Twitter/X** — seguí a [@AnthropicAI](#) y a Dario Amodei, Mike Krieger, Catherine Olsson.

12. Cierre

Esta es la última clase del cuatrimestre que armamos como **workshop** dedicado a Claude. Llegan a esa clase con este PDF leído y con experiencia propia probando cosas.

En el workshop vamos a:

- Resolver una práctica real de la materia con Claude Code en vivo.
- Comparar enfoques (con y sin agente) lado a lado.
- Mostrar MCPs útiles para alumnos de Negocios Digitales.
- Discutir casos reales donde Claude falló y cómo lidiar con eso.
- Armar el "kit de prompts" para que se lleven al primer trabajo.

Antes del workshop, tarea: **probá Claude Code resolviendo una práctica vieja** de la materia. Llegá con dudas concretas.

Si querés profundizar antes del workshop, arrancá por Anthropic Academy y los cursos *Claude 101* y *Claude Code in Action*. Son los dos más importantes para esta materia.

Ignacio Aracena + Juan Costa

Profesores de tutoriales · Analítica de Datos · UdeSA · Otoño 2026

Cualquier corrección o sugerencia sobre este PDF la reciben en clase. Este material va a evolucionar — el mundo de IA cambia cada trimestre.